

Università degli Studi di Napoli Federico II

Corso di Laurea in Ingegneria Informatica

Corso di Ingegneria del Software

Prof. A.R. Fasolino A.A. 2025–2026

ELABORATO DI INGEGNERIA DEL SOFTWARE

“SISTEMA SOFTWARE MATEMATIKA”

Studentessa

Izaro Cuerdo Larraza

Indice

1	Specifiche Informali	3
2	Analisi e Specifica dei Requisiti	4
2.1	Analisi nomi-verbi	4
2.2	Revisione dei requisiti	5
2.3	Glossario dei termini	6
2.4	Classificazione dei requisiti	7
2.4.1	Requisiti funzionali	8
2.4.2	Requisiti sui dati	8
2.4.3	Requisiti Non Funzionali di Qualità	9
2.4.4	Vincoli / Altri requisiti	9
2.5	Modellazione dei casi d'uso	9
2.5.1	Attori e casi d'uso	9
2.5.2	Diagramma dei casi d'uso	11
2.5.3	Scenari	11
2.6	Diagramma delle classi	12
2.6.1	Responsabilità	13
2.7	Diagrammi di sequenza	14
2.7.1	ValutaGrafoAttivo	14
3	Piano di Test Funzionale	15
3.1	ValutaGrafoAttivo	15
3.2	Vincoli (Constraints)	16
3.3	Test Case Derivati	16
4	Progettazione	17
4.1	Diagramma delle Classi	17
4.1.1	Traduzione classi ed associazioni	17
4.1.2	Pattern BCED	18
4.2	Diagrammi di sequenza	20
5	Implementazione	20
5.1	Package Database	21
5.2	Package Entity	22
5.3	Package Controller	23
5.4	Package Boundary	24
5.5	Diagramma di Deployment	25
6	Testing	25
6.1	Test Strutturale	26
6.1.1	Complessità Ciclomantica	26
6.2	Vitest - Test di Unità	28
6.3	Test funzionale	29

1 Specifiche Informali

Il progetto **Matematika** nasce dall'esigenza di modernizzare lo studio della geometria e della logica matematica. Tradizionalmente, lo studio dei teoremi avviene tramite la lettura di libri di testo in modo lineare e passivo. Questo approccio rende difficile comprendere come un concetto logico dipenda strettamente da un altro.

Per risolvere questo problema, è stata ideata e sviluppata un'applicazione web interattiva di tipo *Single Page Application (SPA)*. Matematika rappresenta la conoscenza matematica non come un testo piatto, ma come un **grafo diretto aciclico (DAG)** in cui ogni nodo è un'entità (ad esempio, una Definizione, un Assioma o un Teorema). Di ogni nodo si vuole memorizzare l'identificativo, il tipo e le dipendenze dirette verso altri nodi. Ciascun Teorema può includere una o più Prove (dimostrazioni), dotate ciascuna di un proprio identificativo e delle proprie dipendenze logiche necessarie alla validità della dimostrazione.

Il sistema distingue due profili d'uso principali. Lo **Studente** è il fruitore dei contenuti matematici: esplora il grafo della conoscenza e interagisce con i diagrammi. L'**Autore** è il creatore dei contenuti: può aggiungere, modificare e rimuovere contenuti matematici tramite un Editor dedicato.

Le funzionalità e le caratteristiche principali del sistema includono:

- **Pagina Principale e Rami della Matematica:** Lo Studente viene accolto da una panoramica generale che suddivide la materia nelle sue branche principali, facilitando l'orientamento all'interno del dominio di studio.
- **Esplorazione Non Lineare:** È possibile navigare liberamente tra i concetti tramite una mappa interattiva o un motore di ricerca rapido (Omnibar).
- **Glossario Interattivo e Marginalia:** Il sistema è dotato di un glossario dedicato per la consultazione rapida dei termini matematici. I significati sono accessibili sia tramite una pagina dedicata, sia cliccando direttamente su parole chiave collegate (link) o su formule matematiche, le quali aprono delle spiegazioni laterali (marginalia) senza interrompere il flusso di lettura.
- **Grafo degli Assiomi: Modalità Sandbox e Validazione in Tempo Reale:** Il cuore dell'applicazione è la rappresentazione delle dipendenze matematiche tramite un grafo interattivo. Tramite una speciale Modalità Sandbox, lo Studente può esplorare il grafo accendendo (abilitando) o spegnendo (disabilitando) a piacimento gli assiomi fondamentali. A ogni variazione, un motore di validazione in tempo reale ricalcola l'ordinamento topologico dell'intero sistema. I risultati si riflettono immediatamente sull'interfaccia: se un teorema perde il supporto logico delle sue premesse, viene visibilmente invalidato; se lo acquisisce, viene validato. In fase di inserimento di nuovi contenuti, il sistema verifica l'assenza di cicli nel grafo e, qualora rilevi una dipendenza ciclica, interrompe l'operazione di salvataggio emettendo un errore.
- **Dimostrazioni Dinamiche:** Le pagine di teoria contengono diagrammi geometrici generati dal motore JSXGraph. Tali grafiche reagiscono in tempo reale alle manipolazioni dell'utente.
- **Modelli Matematici e Sistemi Assiomatici:** Il sistema rappresenta modelli matematici concreti (istanze di sistemi assiomatici) e sistemi assiomatici astratti.

Di ogni Modello e Sistema Assiomatico si vuole memorizzare l'identificativo, il titolo e l'elenco degli assiomi che lo compongono o che esso soddisfa.

- **Editor dei Contenuti:** L'Autore dispone di un ambiente di composizione integrato per aggiungere, modificare e rimuovere contenuti matematici (Teoremi, Definizioni, Assiomi, Dimostrazioni), definirne i metadati e stabilirne le dipendenze logiche con i nodi preesistenti del grafo.

In sintesi, Matematika mira a trasformare l'apprendimento della teoria da un esercizio di memoria a un'esperienza di scoperta logica, combinando il rigore dell'Ingegneria del Software con un'interfaccia utente moderna ed educativa.

2 Analisi e Specifica dei Requisiti

2.1 Analisi nomi-verbi

Il progetto Matematika nasce dall'esigenza di **modernizzare lo studio della geometria e della logica matematica**. Tradizionalmente, lo studio dei **teoremi** avviene tramite la lettura di libri di testo in modo lineare e passivo. Questo approccio rende difficile comprendere come un **concetto** logico dipenda strettamente da un altro.

Per risolvere questo problema, è stata **ideata e sviluppata un'applicazione web interattiva** di tipo **Single Page Application (SPA)**. Matematika **rappresenta la conoscenza matematica** non come un testo piatto, ma come un **grafo diretto aciclico (DAG)** in cui ogni **nodo** è un'entità (ad esempio, una **Definizione**, un **Assioma** o un **Teorema**). Di ogni **nodo** si vuole memorizzare l'**identificativo**, il **tipo** e le **dipendenze** dirette verso altri **nodi**. Ciascun **Teorema** può includere una o più **Prove** (dimostrazioni), dotate ciascuna di un proprio **identificativo** e delle proprie **dipendenze** logiche necessarie alla validità della **dimostrazione**.

Il **sistema** distingue due profili d'uso principali. Lo **Studente** è il fruitore dei contenuti matematici: **esplora il grafo della conoscenza** e **interagisce con i diagrammi**. L'**Autore** è il creatore dei contenuti: può **aggiungere**, **modificare** e **rimuovere** contenuti matematici tramite un **Editor** dedicato.

Le **funzionalità** e le caratteristiche principali del **sistema** includono:

- **Pagina Principale e Rami della Matematica:** Lo **Studente** viene accolto da una panoramica generale che **suddivide la materia nelle sue branche principali**, facilitando l'orientamento all'interno del dominio di studio.
- **Esplorazione Non Lineare:** È possibile **navigare liberamente tra i concetti** tramite una **mappa interattiva** o un motore di ricerca rapido (**Omnibar**).
- **Glossario Interattivo e Marginalia:** Il **sistema** è dotato di un **glossario** dedicato per la **consultazione rapida dei termini matematici**. I significati sono accessibili sia tramite una **pagina** dedicata, sia **cliccando direttamente su parole chiave collegate (link)** o su **formule** matematiche, le quali aprono delle spiegazioni laterali (**marginalia**) senza interrompere il flusso di lettura.

- **Grafo degli Assiomi: Modalità Sandbox e Validazione in Tempo Reale:** Il cuore dell'applicazione è la rappresentazione delle dipendenze matematiche tramite un grafo interattivo. Tramite una speciale Modalità Sandbox, lo **Studente** può esplorare il grafo accendendo (abilitando) o spegnendo (disabilitando) a piacimento gli assiomi fondamentali. A ogni variazione, un motore di validazione in tempo reale ricalcola l'ordinamento topologico dell'intero sistema. I risultati si riflettono immediatamente sull'interfaccia: se un teorema perde il supporto logico delle sue premesse, viene visibilmente invalidato; se lo acquisisce, viene validato. In fase di inserimento di nuovi contenuti, il sistema verifica l'assenza di cicli nel grafo e, qualora rilevi una dipendenza ciclica, interrompe l'operazione di salvataggio emettendo un errore.
- **Dimostrazioni Dinamiche:** Le pagine di teoria contengono diagrammi geometrici generati dal motore JSXGraph. Tali grafiche reagiscono in tempo reale alle manipolazioni dell'utente.
- **Modelli Matematici e Sistemi Assiomatici:** Il sistema rappresenta modelli matematici concreti (istanze di sistemi assiomatici) e sistemi assiomatici astratti. Di ogni Modello e Sistema Assiomatico si vuole memorizzare l'identificativo, il titolo e l'elenco degli assiomi che lo compongono o che esso soddisfa.
- **Editor dei Contenuti:** L'Autore dispone di un ambiente di composizione integrato per aggiungere, modificare e rimuovere contenuti matematici (Teoremi, Definizioni, Assiomi, Dimostrazioni), definirne i metadati e stabilirne le dipendenze logiche con i nodi preesistenti del grafo.

In sintesi, Matematika mira a trasformare l'apprendimento della teoria da un esercizio di memoria a un'esperienza di scoperta logica, combinando il rigore dell'Ingegneria del Software con un'interfaccia utente moderna ed educativa.

LEGENDA:

Classe

Attributo

Funzionalità

Attore

Classe-Attore

2.2 Revisione dei requisiti

A seguito di una interazione (simulata) con gli stakeholder, la traccia è stata revisionata per chiarire ambiguità, disambiguare i termini, e separare le frasi che esprimono più requisiti. Di seguito riportiamo i requisiti revisionati, formulati in modo chiaro e verificabile.

1. Il sistema deve offrire allo Studente una funzionalità per esplorare le branche della matematica.
2. Di ogni Nodo si vuole memorizzare identificativo, tipo e dipendenze.

3. Di ogni Prova si vuole memorizzare identificativo e dipendenze.
4. Il sistema deve offrire allo Studente una funzionalità per navigare tra i concetti matematici in modo non lineare.
5. Il sistema deve offrire allo Studente una funzionalità per consultare un glossario interattivo.
6. Il sistema deve offrire allo Studente una funzionalità per modificare lo stato logico degli assiomi.
7. Il sistema deve ricalcolare l'ordinamento topologico in base all'elenco dei nodi.
8. Il sistema deve aggiornare la validità dei teoremi in tempo reale.
9. Il sistema deve verificare l'assenza di cicli in fase di validazione build-time e durante l'inserimento di nuovi contenuti.
10. Il sistema deve invalidare visivamente i nodi dipendenti che perdono le proprie premesse logiche.
11. Il sistema deve validare i nodi dipendenti che le acquisiscono.
12. Il sistema deve interrompere l'operazione di inserimento ed emettere un errore qualora rilevi una dipendenza ciclica.
13. Il sistema deve rappresentare i modelli matematici e i sistemi assiomatici.
14. Di ogni Modello e Sistema Assiomatico si vuole memorizzare identificativo, titolo e l'elenco di Assiomi.
15. Il sistema deve offrire all'Autore una funzionalità per aggiungere contenuti matematici tramite un Editor.
16. Il sistema deve offrire all'Autore una funzionalità per modificare contenuti matematici tramite un Editor.
17. Il sistema deve offrire all'Autore una funzionalità per rimuovere contenuti matematici tramite un Editor.
18. Il sistema deve generare diagrammi geometrici interattivi tramite il motore JSX-Graph che reagiscono in tempo reale alle manipolazioni dell'utente.

2.3 Glossario dei termini

Di seguito si riporta un glossario dei termini principali utilizzati all'interno del dominio applicativo di *Matematika*, al fine di evitare ambiguità e standardizzare il vocabolario del progetto.

Termine	Descrizione	Sinonimi
Grafo	Struttura dati (Grafo Diretto Aciclico o DAG) che modella la dipendenza logica tra i vari concetti matematici.	Rete di conoscenza
Nodo	Elemento fondamentale del grafo che rappresenta una singola entità matematica (es. teorema, assioma, definizione).	Concetto, Entità
Assioma	Proposizione assunta come vera per principio, che funge da punto di partenza (premessa) per dedurre altri risultati.	Postulato
Teorema	Proposizione dimostrata logicamente a partire da assiomi o altri teoremi già dimostrati. Può avere una o più dimostrazioni alternative.	Lemma, Corollario
Lemma	Teorema intermedio dimostrato come passo preliminare per facilitare la dimostrazione di un teorema più complesso.	Teorema ausiliario
Corollario	Conseguenza diretta e immediata di un teorema già dimostrato, che non richiede una dimostrazione estesa.	Conseguenza
Prova	Sequenza logica di deduzioni che valida un Teorema, basandosi sulle proprie dipendenze.	Dimostrazione
Sistema Assiomatico	Insieme di assiomi di base da cui vengono derivati tutti i teoremi in un contesto logico specifico.	Teoria
Modello	Istanza concreta di un sistema assiomatico che soddisfa un insieme specifico di assiomi (es. Piano Cartesiano, Disco di Poincaré).	Istanza, Realizzazione
Sandbox	Modalità interattiva che permette allo Studente di alterare (accendere/spegnere) lo stato logico degli assiomi per osservare i cambiamenti.	Ambiente di test
Studente	Fruitore dei contenuti matematici: esplora il grafo della conoscenza, consulta il glossario e interagisce con i diagrammi.	Discente, Allievo
Marginalia	Spiegazioni aggiuntive che compaiono a margine del testo principale quando l'utente interagisce con un termine o una formula.	Note a margine
Omnibar	Barra unificata che permette la navigazione rapida e globale (ricerca) tra i vari concetti del sistema.	Barra di ricerca
Autore	Creatore di contenuti matematici (es. teoremi, assiomi) tramite Editor dedicato.	Docente, Creatore
Editor	Componente dell'interfaccia che permette all'Autore di creare o modificare nodi, formule e dipendenze.	Ambiente di composizione

Tabella 1: Glossario dei Termini

2.4 Classificazione dei requisiti

Classifichiamo di seguito i requisiti (funzionali, sui dati, vincoli/altri requisiti) riportando le frasi dei requisiti revisionati espresse precedentemente.

2.4.1 Requisiti funzionali

ID	Requisito	Origine (n. frase)
RF01	Il sistema deve offrire allo Studente una funzionalità per esplorare le branche della matematica.	1
RF02	Il sistema deve offrire allo Studente una funzionalità per navigare tra i concetti matematici in modo non lineare.	4
RF03	Il sistema deve offrire allo Studente una funzionalità per consultare un glossario interattivo.	5
RF04	Il sistema deve offrire allo Studente una funzionalità per modificare lo stato logico degli assiomi.	6
RF05	Il sistema deve ricalcolare l'ordinamento topologico in base all'elenco dei nodi.	7
RF06	Il sistema deve aggiornare la validità dei teoremi in tempo reale.	8
RF07	Il sistema deve verificare l'assenza di cicli in fase di validazione build-time e durante l'inserimento di nuovi contenuti.	9
RF08	Il sistema deve invalidare visivamente i nodi dipendenti che perdono le proprie premesse logiche.	10
RF09	Il sistema deve validare i nodi dipendenti che le acquisiscono.	11
RF10	Il sistema deve interrompere l'operazione di inserimento ed emettere un errore qualora rilevi una dipendenza ciclica.	12
RF11	Il sistema deve rappresentare i modelli matematici e i sistemi assiomatici.	13
RF12	Il sistema deve offrire all'Autore una funzionalità per aggiungere contenuti matematici tramite un Editor.	15
RF13	Il sistema deve offrire all'Autore una funzionalità per modificare contenuti matematici tramite un Editor.	16
RF14	Il sistema deve offrire all'Autore una funzionalità per rimuovere contenuti matematici tramite un Editor.	17
RF15	Il sistema deve generare diagrammi geometrici interattivi tramite il motore JSXGraph che reagiscono in tempo reale alle manipolazioni dell'utente.	18

2.4.2 Requisiti sui dati

ID	Requisito	Origine (n. frase)
RD01	Di ogni Nodo si vuole memorizzare identificativo, tipo e dipendenze.	2
RD02	Di ogni Prova si vuole memorizzare identificativo e dipendenze.	3
RD03	Di ogni Modello e Sistema Assiomatico si vuole memorizzare identificativo, titolo e l'elenco di Assiomi.	14

2.4.3 Requisiti Non Funzionali di Qualità

ID	Categoria	Descrizione	Metrica	Valore atteso	Verifica
RQ QUAL-01	Prestazioni	Il sistema deve ricalcolare l'intero grafo topologico rapidamente.	Tempo di esecuzione	≤ 1 secondo	Test di performance
RQ QUAL-02	Usabilità	L'interfaccia deve aggiornarsi in tempo reale senza ricaricare la pagina.	Refresh di pagina	0 (zero) refresh	Test manuale UI
RQ QUAL-03	Mantenibilità	Il sistema deve impedire il commit di codice con riferimenti incrociati non validi tra i file MDX.	Commit bloccati con link rotti	0 (zero)	Hook pre-commit Husky
RQ QUAL-04	Affidabilità	I canvas grafici (JSX-Graph, Three.js) devono essere purgati dalla memoria al cambio di rotta.	Memory leak dopo navigazione	0 KB di crescita dopo 50 navigazioni	Memory profiler browser
RQ QUAL-05	Mantenibilità	Il sistema deve convalidare la struttura del grafo logico a ogni build, bloccando la compilazione in caso di errori.	Build fallite per grafo non valido	0 (zero) build con grafo corrotto	Script <code>validate-logical-graph.ts</code>

2.4.4 Vincoli / Altri requisiti

ID	Requisito	Origine
V01	L'applicazione deve essere compatibile con i principali browser web moderni (Chrome, Firefox, Safari).	-
V02	Il sistema è implementato come Single Page Application (SPA) utilizzando React e TypeScript.	-

2.5 Modellazione dei casi d'uso

2.5.1 Attori e casi d'uso

Attori Primari:

- Studente
- Autore

Attori Secondari:

- -

Casi d'uso:

- UC1: EsploraBranche

- UC2: NavigaConcetti
- UC3: ConsultaGlossario
- UC4: EsploraModelli
- UC5: VisualizzaDimostrazione
- UC6: ValutaGrafoAttivo (Modalità Sandbox)
- UC7: AggiungiContenuto
- UC8: ModificaContenuto
- UC9: RimuoviContenuto

Casi d'uso di inclusione:

- UC10: RicalcolaOrdinamento
- UC11: AggiornaValidita
- UC12: VerificaCicli

Caso d'uso	Attori Primari	Attori Secondari	Incl. / Ext.	Req. corr.
UC1: EsploraBranche	Studente	-	-	RF01
UC2: NavigaConcetti	Studente	-	-	RF02
UC3: ConsultaGlossario	Studente	-	-	RF03
UC4: EsploraModelli	Studente	-	-	RF11
UC5: VisualizzaDimostrazione	Studente	-	-	RF15
UC6: ValutaGrafoAttivo	Studente	-	Include UC10, UC11	RF04
UC7: AggiungiContenuto	Autore	-	Include UC12	RF12
UC8: ModificaContenuto	Autore	-	-	RF13
UC9: RimuoviContenuto	Autore	-	-	RF14
UC10: RicalcolaOrdinamento	-	-	Incluso in UC6	RF05
UC11: AggiornaValidita	-	-	Incluso in UC6	RF06, RF08, RF09
UC12: VerificaCicli	-	-	Incluso in UC7	RF07, RF10

2.5.2 Diagramma dei casi d'uso

Questo diagramma mostra come gli attori (lo Studente e l'Autore) interagiscono con i casi d'uso principali del sistema Matematika.

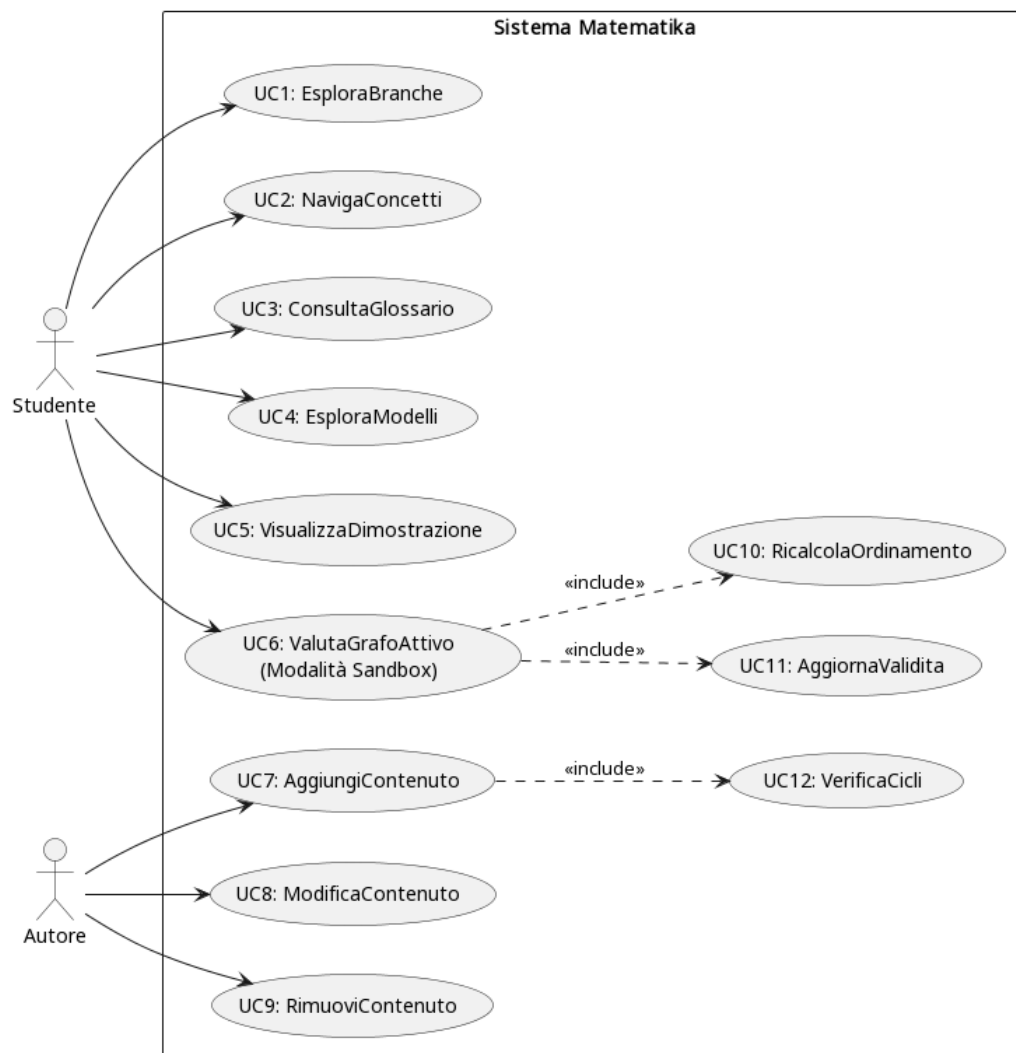


Figura 1: Diagramma dei Casi d'Uso

2.5.3 Scenari

Viene di seguito riportato lo scenario principale per il caso d'uso scelto, che verrà sviluppato fino alla codifica.

Attributo	Descrizione
Caso d'uso	ValutaGrafoAttivo
Attore primario	Studiante
Attore secondario	-
Descrizione	Lo Studente modifica lo stato di verità di uno o più assiomi. Il sistema ricalcola dinamicamente l'ordinamento topologico e aggiorna lo stato di validità di tutti i teoremi dipendenti.
Pre-Condizioni	Lo Studente si trova nell'ambiente Sandbox del Grafo. Il grafo matematico è istanziato in memoria.
Sequenza di eventi principale	1. Il caso d'uso inizia quando lo Studente accede alla Modalità Sandbox del Grafo. 2. Lo Studente seleziona un assioma tramite l'interfaccia. 3. Lo Studente inverte lo stato logico dell'assioma (da acceso a spento o viceversa). 4. Il sistema esegue <<include>> RicalcolaOrdinamento, attraversando il grafo in ordine topologico pre-validato e assumendo un DAG aciclico. 5. Il sistema esegue <<include>> AggiornaValidita, aggiornando lo stato di validità dei nodi dipendenti in base al risultato del ricalcolo. 6. Il sistema aggiorna lo stato visivo dei nodi nell'interfaccia: i teoremi che hanno perso il supporto delle premesse appaiono invalidati; quelli che lo hanno acquisito appaiono validati.
Post-Condizioni	L'interfaccia riflette la nuova conformazione logica del grafo.
Casi d'uso correlati	RicalcolaOrdinamento, AggiornaValidita
Sequenza di eventi alternativa	Al punto 4, se il grafo statico presenta dati inconsistenti (es. nodi mancanti nell'indice), il sistema ignora silenziosamente il nodo problematico e prosegue la valutazione con i dati disponibili.

Tabella 2: Specifica del Caso d'Uso: ValutaGrafoAttivo

2.6 Diagramma delle classi

Sviluppo del Diagramma delle classi di analisi corrispondente al System Domain Model. Esso mostra i concetti reali (le entità matematiche) e le loro relazioni, rappresentando il "cuore" logico del software.

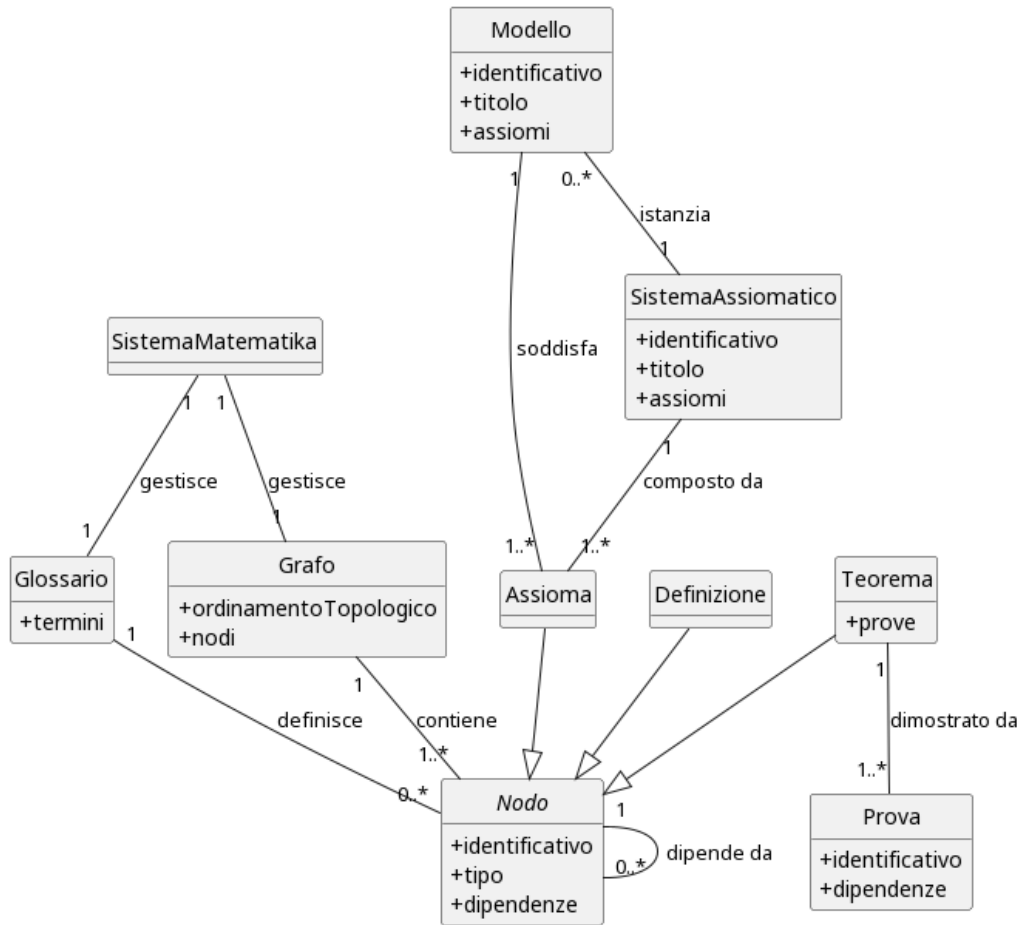


Figura 2: Diagramma delle Classi di Analisi (Modello di Dominio)

Il diagramma evidenzia come **SistemaMatematika** agisca da entry point (Facade), orchestrando le operazioni sul **Grafo** e sul **Glossario**. I termini evidenziati nell'analisi nomi-verbi che non compaiono in questo modello — quali **Omnibar**, **Marginalia**, **Editor**, **Mappa Interattiva** e **JSXGraph** — rappresentano componenti dell'interfaccia utente, pattern di presentazione o librerie esterne, la cui definizione architetturale è rinviata al Capitolo 4 (Pattern BCED). Il termine **Modalità Sandbox**, invece, denota un modo di interazione con il **Grafo**, non un'entità autonoma.

2.6.1 Responsabilità

La seguente tabella associa a ciascuna responsabilità principale del sistema la classe del Modello di Dominio che ne è responsabile, secondo il principio *Information Expert* (GRASP).

Responsabilità	Classe
EsploraBranche	SistemaMatematika
NavigaConcetti	Grafo
ConsultaGlossario	Glossario
EsploraModelli	Modello
VisualizzaDimostrazione	SistemaMatematika
ValutaGrafoAttivo	Grafo
AggiungiContenuto	SistemaMatematika
ModificaContenuto	Nodo
RimuoviContenuto	Grafo
RicalcolaOrdinamento	Grafo
AggiornaValidita	Grafo
VerificaCicli	Grafo

Tabella 3: Responsabilità delle Classi del Modello di Dominio

EsploraBranche, AggiungiContenuto e VisualizzaDimostrazione sono responsabilità di SistemaMatematika, in quanto *Controller* che orchestra le richieste.

NavigaConcetti, ValutaGrafoAttivo, RicalcolaOrdinamento, AggiornaValidita e VerificaCicli sono responsabilità di Grafo, in quanto *Information Expert* della topologia del DAG.

RimuoviContenuto è responsabilità di Grafo, in quanto *Creator* e gestore del ciclo di vita dei nodi.

ConsultaGlossario è responsabilità di Glossario, in quanto *Information Expert* dei termini.

EsploraModelli è responsabilità di Modello, in quanto *Information Expert* degli assiomi soddisfatti.

ModificaContenuto è responsabilità di Nodo, in quanto *Information Expert* dei propri metadati e dipendenze.

2.7 Diagrammi di sequenza

Viene qui riportato il diagramma di sequenza di analisi (System Sequence Diagram) per il caso d'uso scelto. In questa fase, il sistema è rappresentato come una *black box*: i messaggi mostrano esclusivamente l'interazione tra l'attore e il sistema nel suo complesso, senza dettagli architetturali interni (che saranno introdotti nel Capitolo 4).

2.7.1 ValutaGrafoAttivo

In questo scenario, lo **Studiante** accede alla Modalità Sandbox e il sistema risponde mostrando l'elenco degli assiomi disponibili. Lo Studiante modifica lo stato di un assioma con un'unica azione di toggle. Il sistema, rappresentato come un'unica entità, esegue internamente il ricalcolo dell'ordinamento topologico e l'aggiornamento della validità, quindi restituisce il risultato aggiornando l'interfaccia.

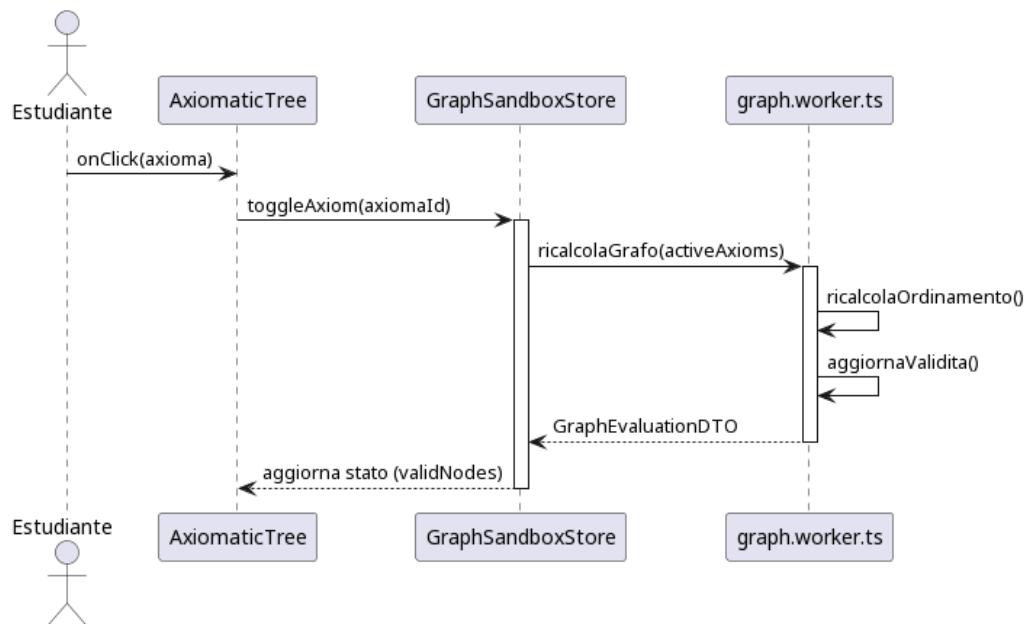


Figura 3: Diagramma di Sequenza di Analisi: ValutaGrafoAttivo

3 Piano di Test Funzionale

In questo capitolo viene definito il piano di test funzionale per il caso d'uso **UC6: ValutaGrafoAttivo**, selezionato per lo sviluppo fino alla codifica. Utilizzando il metodo *Category Partition Testing* (CPT), si identificano i parametri e le categorie di equivalenza della funzione principale **Grafo.evaluate()**, responsabile del ricalcolo dell'ordinamento topologico e della determinazione della validità dei nodi del grafo. Dai parametri categorizzati vengono derivati i test case che costituiscono la test suite.

3.1 ValutaGrafoAttivo

Applichiamo il metodo *Category Partition Testing* alla funzione **Grafo.evaluate()**. Sono stati identificati i seguenti parametri e categorie di equivalenza:

Assiomi Attivi	Tipo di Dipendenza	Integrità del Grafo
A1: Nessun assioma attivo — insieme vuoto.	D1: Prova con dipendenze — il Teorema è validato da una dimostrazione che dipende da più nodi.	I1: Integro — tutti i nodi dell'ordine topologico esistono nell'indice.
A2: Assiomi parziali — solo alcuni assiomi della catena sono attivi.	D2: Prove multiple (OR) — il Teorema dispone di più dimostrazioni alternative.	I2: Nodi mancanti — l'ordine topologico contiene ID non presenti nell'indice.
A3: Assiomi completi — tutti gli assiomi necessari per la catena sono attivi.	D3: Nessuna dipendenza — il nodo non ha dipendenze né prove (caso base).	

Tabella 4: Category Partition: ValutaGrafoAttivo

3.2 Vincoli (Constraints)

Nella generazione della Test Suite, per evitare un'esplosione combinatoria ($3 \times 3 \times 2 = 18$ combinazioni possibili) e rimuovere test irrilevanti o ridondanti, sono stati individuati i seguenti vincoli (*Constraints*) tra le categorie:

- **C1 [Error / Single]:** La categoria I2 (Nodi mancanti) rappresenta uno stato anomalo dell'ambiente. Viene testata con un'unica combinazione rappresentativa (A3) ignorando il tipo di dipendenza, per verificare la robustezza del ciclo di attraversamento.
- **C2 [If]:** La categoria D3 (Nessuna dipendenza) determina la validità a priori del nodo. Viene testata con A1 (Nessun assioma attivo) in modo da confermare che tali nodi diventino validi anche nel caso peggiore.
- **C3 [Property]:** La categoria A2 (Assiomi parziali) è l'unica condizione che permette di testare la corretta implementazione della logica OR tra prove multiple (D2) e l'interruzione di una catena singola (D1).

3.3 Test Case Derivati

Dalla combinazione delle categorie soggette ai vincoli, sono stati derivati 5 test case ottimali, ciascuno mirato a coprire un comportamento logico specifico della funzione.

TC ID	Descrizione	Classi eq. coperte	Pre-condizioni	Input	Output attesi	Post-condizioni
TC_01	Catena completa valida	A3, D1, I1	Grafo integro.	Tutti gli assiomi della catena attivi.	Tutti i nodi della catena risultano validi. Il teorema è validato da una Prova che dipende da più nodi, tutti soddisfatti dagli assiomi attivi.	L'insieme dei nodi validi contiene l'intera catena.
TC_02	Nessun assioma attivo	A1, D3, I1	Grafo integro.	Insieme vuoto di assiomi attivi.	Solo i teoremi senza dipendenze né prove risultano validi.	L'insieme dei nodi validi è vuoto o contiene solo nodi senza dipendenze.
TC_03	Dipendenza interrotta	A2, D1, I1	Grafo integro.	Attivo un assioma da cui la definizione NON dipende.	L'assioma attivo è valido; la definizione con dipendenza non soddisfatta non è valida.	I nodi a valle della dipendenza interrotta restano non validi.
TC_04	Logica OR tra prove	A2, D2, I1	Grafo integro. Teorema con due prove alternative.	Attivo un assioma che soddisfa solo una delle due prove.	Il teorema risulta valido (basta una prova soddisfatta).	Il teorema è incluso nell'insieme dei nodi validi.
TC_05	Nodo mancante nell'indice	A3, I2	Ordine topologico contiene un ID non presente nell'indice dei nodi.	Tutti gli assiomi della catena attivi.	Il nodo mancante viene ignorato; la valutazione prosegue con i dati disponibili.	Nessun errore; il sistema completa la valutazione.

Tabella 5: Test Suite: ValutaGrafoAttivo

4 Progettazione

4.1 Diagramma delle Classi

4.1.1 Traduzione classi ed associazioni

Il passaggio dall'analisi alla progettazione comporta la traduzione delle classi concettuali del Modello di Dominio (§2.6) in classi di progetto organizzate secondo il pattern architetturale **BCED (Boundary-Control-Entity-Database)**. La tabella seguente riporta la corrispondenza tra le classi di analisi e i package di progettazione.

Classe di Analisi	Package BCED	Classe di Progetto
SistemaMatematika	Boundary, Controller	AxiomSandboxPanel, GraphSandboxStore
Grafo	Entity	Grafo
Nodo	Entity	Nodo, GraphNodeMeta (in graphTypes)
Teorema, Assioma, Definizione	Entity	Nodo
Prova	Entity	GraphNodeProof (in graphTypes)
Glossario	Database, Controller, Boundary	glossaryDictionary, GlossaryStore, DictionaryPage
Modello, SistemaAssiomatico	Entity	ModelInfo, SystemInfo (in graphTypes)
—	Database	ContentStore, graph_structure.json

Tabella 6: Traduzione delle Classi di Analisi in Classi di Progetto

4.1.2 Pattern BCED

L'architettura di Matematika adotta il pattern BCED, adattato al paradigma React/TypeScript: le classi Boundary sono componenti React, i Controller sono store Zustand, le Entity sono classi TypeScript con logica di dominio, e il Database è costituito da file JSON e MDX accessibili tramite un DAO.

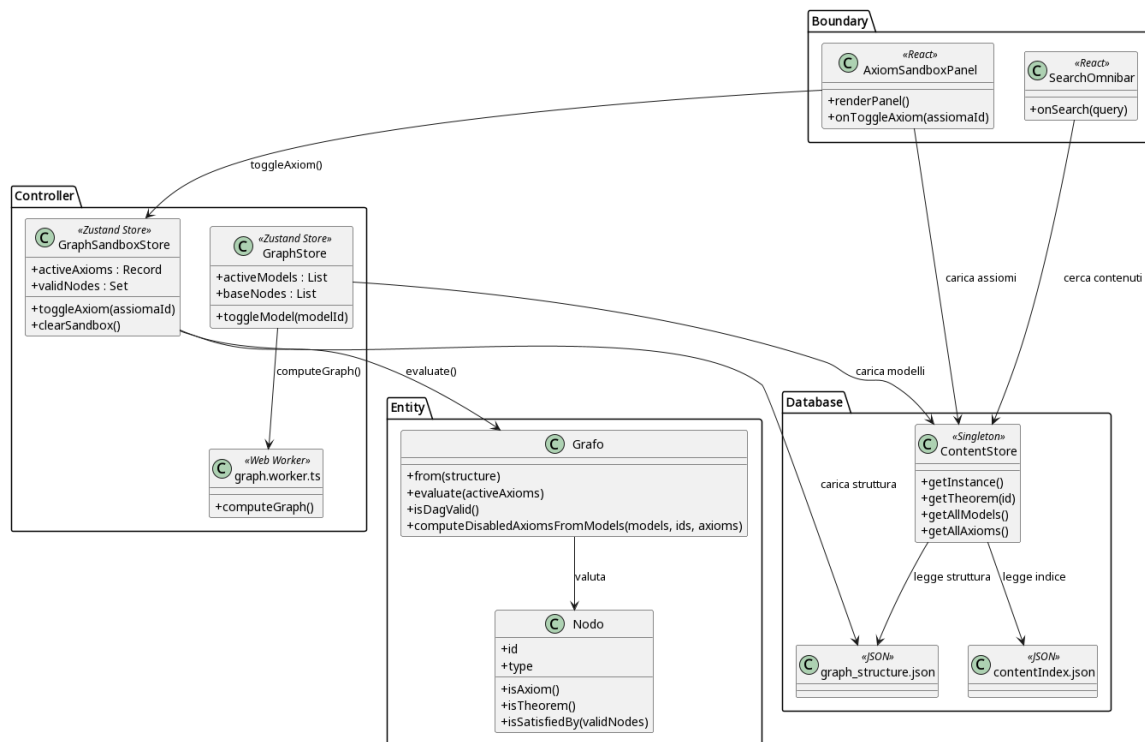


Figura 4: Diagramma delle Classi di Progettazione (BCED)

4.1.2.1 Package Boundary

Il package Boundary (`src/boundary/`) contiene i componenti React che gestiscono l'interfaccia utente. I componenti principali sono:

- **AxiomSandboxPanel** (`components/ui/`) — pannello della Modalità Sandbox: mostra gli assiomi disponibili e invia l'evento `toggleAxiom()` al Controller.
- **TheoremPage**, **DefinitionPage**, **AxiomPage**, **ModelPage** (`pages/`) — pagine di visualizzazione dei contenuti matematici.
- **SearchOmnibar** (`components/navigation/`) — barra di ricerca globale con filtro per tipo di contenuto.
- **MarginaliaPanel** (`components/content/`) — pannello laterale per la consultazione contestuale dei termini.
- **EditorPage** (`pages/Editor/`) — editor MDX per la creazione e modifica di contenuti matematici.

4.1.2.2 Package Controller

Il package Controller (`src/controller/`) contiene gli store Zustand che orchestrano la logica applicativa. I controller principali sono:

- **GraphSandboxStore** (`store/`) — gestisce lo stato della Modalità Sandbox: attivazione/disattivazione degli assiomi e validazione in tempo reale tramite `Grafo.evaluate()`.
- **GraphStore** (`store/`) — gestisce il grafo globale: modelli attivi, sistemi assiomatici, nodi ed edge renderizzati. Delega i calcoli asincroni al Web Worker `graph.worker.ts`.
- **GlossaryStore**, **NavigationStore**, **UserProgressStore**, **MathStore**, **DynamicVarStore** (`store/`) — gestiscono rispettivamente il glossario, la navigazione, il progresso dell'utente, lo stato matematico globale e le variabili dinamiche per i contenuti interattivi.

4.1.2.3 Package Entity

Il package Entity (`src/entity/`) contiene le classi che rappresentano i concetti del dominio matematico e incapsulano la logica di business:

- **Grafo** (`Grafo.ts`) — classe che incapsula la struttura del DAG. Fornisce il metodo `evaluate()` per la validazione topologica, `isDagValid()` per la verifica di integrità, e il metodo statico `computeDisabledAxiomsFromModels()`.
- **Nodo** (`Nodo.ts`) — value object che rappresenta un nodo del grafo. Fornisce `isSatisfiedBy()`, `isAxiom` e `isTheorem` per determinare la validità logica del nodo.
- **graphTypes** (`graphTypes.ts`) — definisce le interfacce TypeScript del dominio: `GraphStructure`, `GraphNodeMeta`, `GraphNodeProof`, `ModelInfo`, `SystemInfo`, `GraphEdge`.

4.1.2.4 Package Database

Il package Database (`src/database/`) gestisce la persistenza. Poiché Matematika è una SPA statica, non utilizza un DBMS relazionale ma file JSON e MDX:

- `ContentStore` (`dao/content/`) — DAO con pattern Singleton: carica, valida (tramite `schemas.ts` con Zod) e indicizza i contenuti MDX.
- `graph_structure.json` — struttura pre-calcolata del DAG con ordinamento topologico e metadati dei nodi.
- `contentIndex.json` — indice generato a build-time di tutti i contenuti.
- `schemas.ts` — schemi Zod per la validazione dei metadati dei contenuti MDX.

4.2 Diagrammi di sequenza

Il diagramma di sequenza di progettazione per il caso d'uso **UC6: ValutaGrafoAttivo** mostra l'interazione tra i componenti del package BCED. Lo Studente interagisce con la Boundary (`AxiomSandboxPanel`), che invia l'evento al Controller (`GraphSandboxStore`). Il Controller aggiorna lo stato degli assiomi attivi e delega la valutazione logica all'Entity (`Grafo`), che itera sui nodi in ordine topologico verificando la soddisfacibilità di ciascun Nodo. Il risultato della valutazione viene propagato dal Controller alla Boundary, che aggiorna reattivamente l'interfaccia.

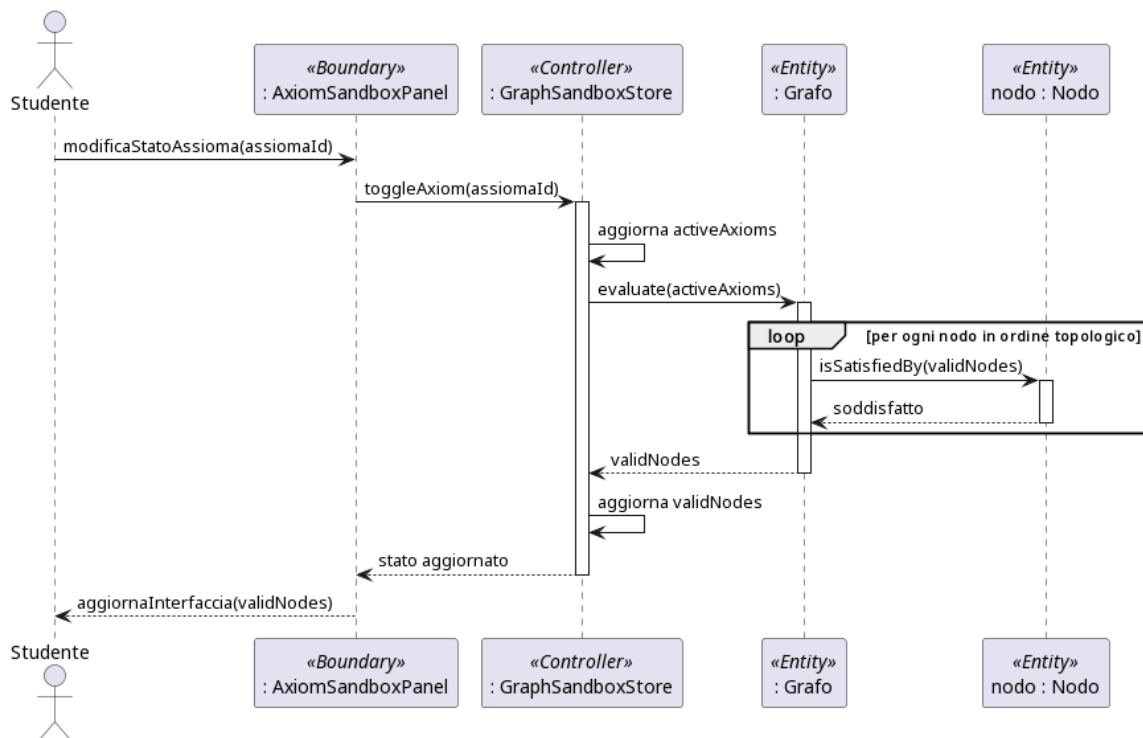


Figura 5: Diagramma di Sequenza di Progettazione: ValutaGrafoAttivo

5 Implementazione

L'applicazione è stata sviluppata in ambiente Node.js utilizzando TypeScript, React 19 e Vite come bundler. La gestione dello stato è affidata a Zustand, lo styling a Tailwind

CSS, e la validazione dei dati a Zod. I contenuti matematici sono scritti in formato MDX e compilati a build-time. In questa sezione vengono presentate le scelte implementative relative ai package dell'architettura BCED.

5.1 Package Database

Il package Database (`src/database/`) fornisce i dati all'applicazione. Poiché *Matematika* è una SPA statica, non utilizza un DBMS relazionale ma un database basato su file JSON pre-compilati e file MDX. Il ruolo di *Data Access Object (DAO)* è svolto dalla classe `ContentStore` (`dao/content/ContentStore.ts`), che centralizza l'accesso ai contenuti e implementa il pattern Singleton. La classe mantiene in memoria mappe indicizzate per ciascun tipo di contenuto (teoremi, definizioni, assiomi, modelli, etc.) e li carica e valida tramite gli schemi Zod definiti in `schemas.ts`. Si riporta di seguito uno stralcio significativo della classe:

```

1 export class ContentStore {
2   // Mappe indicizzate per tipo di contenuto
3   public theorems: Map<string, Theorem> = new Map();
4   public definitions: Map<string, Definition> = new Map();
5   public axioms: Map<string, Axiom> = new Map();
6   public models: Map<string, Model> = new Map();
7   // ... altre mappe per lessons, demos, mathematicians, etc.
8
9   constructor() {
10    this.init(); // Carica e valida tutti i contenuti all'avvio
11  }
12
13  private init(): void {
14    // Itera sui loader MDX generati a build-time
15    for (const loader of contentLoaders) {
16      const { slug, frontmatter, default: Component } = loader;
17      const entry = contentIndex[slug];
18      if (!entry) continue;
19      // Validazione del contenuto con lo schema Zod corrispondente
20      const schema = this.getSchema(entry.contentType);
21      if (schema) {
22        const result = schema.safeParse(frontmatter);
23        if (!result.success) continue; // Contenuto non valido, ignorato
24      }
25      // Archivia il contenuto validato nella mappa corrispondente
26      this.store(entry.contentType, slug, frontmatter, Component);
27    }
28  }
29 }

```

La struttura dei dati persistenti è definita in due file JSON principali:

- `contentIndex.json` — indice generato a build-time (`generate-content-index.ts`) che mappa ogni slug ai metadati del contenuto.
- `graph_structure.json` — grafo delle dipendenze logiche con ordinamento topologico pre-calcolato, utilizzato dall'entità *Grafo* per la validazione in tempo reale.

5.2 Package Entity

Il package Entity (src/entity/) contiene le classi che rappresentano i concetti del dominio matematico e incapsulano la logica di business. È composto da tre file:

- **Grafo.ts** — classe che incapsula la struttura del DAG. Fornisce il metodo `evaluate()` per la validazione topologica dei nodi, `isDagValid()` per la verifica di integrità strutturale, e il metodo statico `computeDisabledAxiomsFromModels()` per determinare quali assiomi disabilitare in base ai modelli matematici attivi.
- **Nodo.ts** — value object che rappresenta un singolo nodo del grafo. Espone le proprietà `isAxiom` e `isTheorem` e il metodo `isSatisfiedBy(validNodes)` che determina se il nodo è logicamente valido dato l'insieme corrente di nodi attivi, implementando la logica OR per i teoremi con dimostrazioni multiple.
- **graphTypes.ts** — interfacce TypeScript (`GraphStructure`, `GraphNodeMeta`, `GraphNodeProof`, `ModelInfo`, `SystemInfo`, `GraphEdge`) che definiscono i tipi del dominio.

Si riporta di seguito il metodo principale `evaluate()` della classe `Grafo`, responsabile del ricalcolo dell'ordinamento topologico e della validità dei nodi:

```

1 export class Grafo {
2   private readonly order: string[];           // Ordine topologico pre-
      calculato
3   private readonly nodi: ReadonlyMap<string, Nodo>; // Mappa dei nodi
4
5   // Factory method: costruisce il grafo a partire dalla struttura JSON
6   static from(structure: GraphStructure): Grafo {
7     return new Grafo(structure);
8   }
9
10  // Valida la struttura DAG e restituisce i nodi logicamente validi
11  evaluate(activeAxioms: Record<string, boolean>): Set<string> {
12    const validNodes = new Set<string>();
13
14    // 1. Inizializza i nodi validi con gli assiomi attivi
15    for (const [axiomId, isActive] of Object.entries(activeAxioms)) {
16      if (isActive) validNodes.add(axiomId);
17    }
18
19    // Solo i tipi visibili nel grafo vengono valutati
20    const VISIBLE = new Set([
21      'assioma', 'lema', 'corolario', 'teorema', 'definicion', 'modelo'
22    ]);
23
24    // 2. Valuta i nodi seguendo l'ordine topologico
25    for (const nodeId of this.order) {
26      if (validNodes.has(nodeId)) continue; // Nodo gi valido
27      const nodo = this.nodi.get(nodeId);
28      if (!nodo) continue;                 // Nodo assente nell'indice
29      if (nodo.isAxiom) continue;          // Gi gestito nel passo 1
30      if (!VISIBLE.has(nodo.type)) continue; // Tipi nascosti ignorati
31
32      // Il nodo valido se tutte le sue dipendenze sono soddisfatte
33      if (nodo.isSatisfiedBy(validNodes)) {
34        validNodes.add(nodeId);
35      }

```

```

36     }
37
38     return validNodes;
39 }
40 }

```

Il metodo itera sui nodi seguendo l'ordine topologico pre-calcolato, garantendo che le dipendenze di ciascun nodo siano state valutate prima del nodo stesso. L'insieme **VISIBLE** filtra i tipi di nodo rilevanti per la validazione (assiomi, lemmi, corollari, teoremi, definizioni e modelli), escludendo tipi ausiliari come sistemi assiomatici, matematici e lezioni. Per i teoremi, la validità è determinata dal metodo `isSatisfiedBy()` della classe `Nodo`, che verifica se almeno una delle dimostrazioni ha tutte le proprie dipendenze soddisfatte (logica OR).

5.3 Package Controller

Il package `Controller` (`src/controller/`) contiene gli store `Zustand` che orchestrano la logica applicativa e coordinano i casi d'uso. Lo store principale per il caso d'uso UC6 è `GraphSandboxStore` (`store/GraphSandboxStore.ts`), che gestisce lo stato della Modalità `Sandbox`. Altri store rilevanti includono `GraphStore` per il grafo globale e `GlossaryStore` per il glossario.

`GraphSandboxStore` carica la struttura del grafo a partire da `graph_structure.json` e istanzia l'entità `Grafo` tramite il factory method `Grafo.from()`. A ogni interazione dell'utente (toggle di un assioma), il `Controller` aggiorna lo stato degli assiomi attivi e delega la validazione all'entità `Grafo.evaluate()`. L'uso di `Zustand` garantisce che i componenti `React` siano reattivamente aggiornati a ogni cambiamento di stato, senza necessità di DTO intermedi.

```

1  // Carica la struttura del grafo a build-time e istanzia l'entità Grafo
2  const staticGraphData = graphStructureData as GraphStructure;
3  const grafo = Grafo.from(staticGraphData);
4
5  export const useGraphSandboxStore = create<GraphSandboxState>((set) =>
6    ({
7      activeAxioms: {},
8      validNodes: new Set<string>(),
9
10     // Metodo principale: attiva/disattiva un assioma e rivaluta il grafo
11     toggleAxiom: (axiomId: string) => {
12       set((state) => {
13         const newAxioms = { ...state.activeAxioms };
14         if (newAxioms[axiomId]) {
15           delete newAxioms[axiomId]; // Disattiva l'assioma
16         } else {
17           newAxioms[axiomId] = true; // Attiva l'assioma
18         }
19         // Delega la validazione all'entità Grafo
20         const newValidNodes = grafo.evaluate(newAxioms);
21         return { activeAxioms: newAxioms, validNodes: newValidNodes };
22       }),
23
24     // Ripristina lo stato iniziale: nessun assioma attivo
25     clearSandbox: () => {
26       const newValidNodes = grafo.evaluate({});
27       set({ activeAxioms: {}, validNodes: newValidNodes });
28     }
29   });

```

```

27   },
28   });

```

Il metodo `toggleAxiom()` implementa il caso d'uso UC6: riceve l'identificativo dell'assioma dalla Boundary, inverte il suo stato logico (da acceso a spento o viceversa), e chiama `grafo.evaluate()` per ricalcolare la validità dell'intero grafo. Il metodo `clearSandbox()` ripristina lo stato iniziale valutando il grafo con un insieme vuoto di assiomi.

5.4 Package Boundary

Il package Boundary (`src/boundary/`) contiene i componenti React che gestiscono l'interfaccia utente. Il componente principale per il caso d'uso UC6 è `AxiomSandboxPanel` (`components/ui/AxiomSandboxPanel.tsx`), un pannello laterale che permette allo Studente di attivare o disattivare gli assiomi e di visualizzare in tempo reale lo stato di validità dei nodi.

Il componente ottiene l'accesso al Controller tramite l'hook Zustand `useGraphSandboxStore()`, da cui destruttura `activeAxioms`, `toggleAxiom`, `validNodes` e altri metodi. I dati degli assiomi e dei modelli sono caricati dal Database tramite il DAO `db` (`ContentStore`).

```

1  export const AxiomSandboxPanel: React.FC = () => {
2    // Accesso al Controller tramite hook Zustand
3    const {
4      activeAxioms, toggleAxiom, validNodes, clearSandbox
5    } = useGraphSandboxStore();
6
7    // Caricamento dati dal Database tramite il DAO ContentStore
8    const models = db.getAllModels();
9    const axioms = db.getAllAxioms();
10
11    return (
12      <div>
13        /* Renderizza una checkbox per ogni assioma */
14        {axioms.map(axiom => {
15          const isActive = !!activeAxioms[axiom.id];
16          return (
17            <label key={axiom.id}>
18              <input
19                type="checkbox"
20                checked={isActive}
21                // Al click, invoca il Controller per il toggle
22                onChange={() => toggleAxiom(axiom.id)}
23              />
24              {axiom.title}
25            </label>
26          );
27        })}
28        /* Footer con statistiche in tempo reale */
29        <footer>
30          Axiomas: {Object.values(activeAxioms).filter(Boolean).length}
31          Conceptos Validos: {validNodes.size}
32        </footer>
33      </div>
34    );
35  };

```


L'interazione con il Controller avviene tramite la callback `onChange`: a ogni click sulla checkbox di un assioma, il Boundary invoca `toggleAxiom(axiom.id)`. Il Controller elabora la richiesta e aggiorna lo stato; Zustand notifica reattivamente il componente, che ri-renderizza l'interfaccia con i nuovi valori di `activeAxioms` e `validNodes`. Il footer del pannello mostra in tempo reale il conteggio degli assiomi attivi e dei concetti validi, fornendo un feedback immediato all'utente.

5.5 Diagramma di Deployment

L'applicazione viene distribuita tramite **GitHub Pages**. A ogni push sul branch `main`, il workflow di GitHub Actions esegue il checkout del repository, l'installazione delle dipendenze e la build tramite il comando `npm run build`, che a sua volta esegue la generazione dell'indice dei contenuti (`generate-content-index.ts`), la validazione del grafo logico (`validate-logical-graph.ts`) e la compilazione del bundle statico con Vite. L'artefatto prodotto (cartella `dist/`) viene caricato come artefatto di pagina e pubblicato tramite `actions/deploy-pages`.

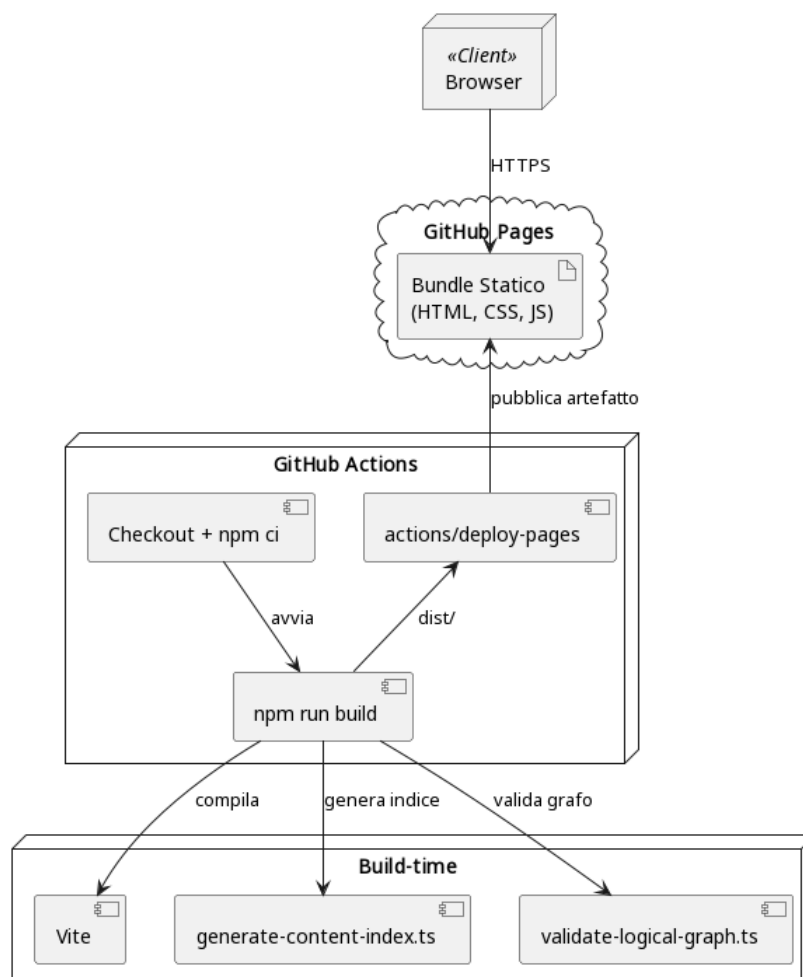


Figura 6: Diagramma di Deployment

6 Testing

6.1 Test Strutturale

6.1.1 Complessità Ciclomatica

È stato analizzato il metodo `evaluate()` della classe `Grafo` (`src/entity/Grafo.ts`), responsabile del ricalcolo topologico e della validazione dei nodi del DAG. Di seguito è riportato il Control Flow Graph (CFG) del metodo.

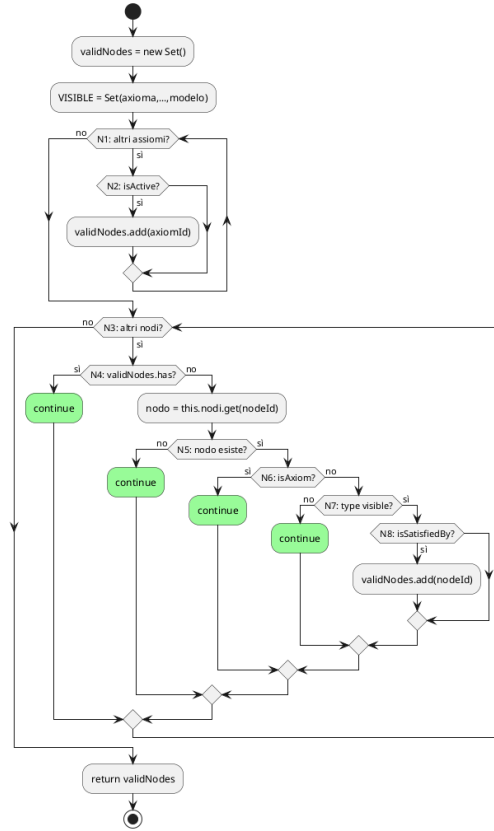


Figura 7: Control Flow Graph (CFG) di `Grafo.evaluate()` con 8 nodi decisionali N1–N8

Il codice sorgente del metodo, con gli 8 nodi decisionali annotati, è il seguente:

```

1  evaluate(activeAxioms: Record<string, boolean>): Set<string> {
2      const validNodes = new Set<string>();
3      const VISIBLE = new Set(['axioma', 'lema', 'corolario', 'teorema',
4                               'definicion', 'modelo']);
5
6      for (const [axiomId, isActive] of Object.entries(activeAxioms)) { // N1
7          if (isActive) // N2
8              validNodes.add(axiomId);
9      }
10
11     for (const nodeId of this.order) { // N3
12         if (validNodes.has(nodeId)) continue; // N4
13         const nodo = this.nodi.get(nodeId);
14         if (!nodo) continue; // N5
15         if (nodo.isAxiom) continue; // N6
16         if (!VISIBLE.has(nodo.type)) continue; // N7
17     }
18     return validNodes;
19 }

```

```

17     if (nodo.isSatisfiedBy(validNodes)) // N8
18         validNodes.add(nodeId);
19     }
20
21     return validNodes;
22 }

```

Il CFG presenta **8 nodi decisionali**: N1 e N3 sono le condizioni di iterazione dei due cicli `for`, mentre N2, N4, N5, N6, N7 e N8 sono condizioni `if` interne. Applicando la formula di McCabe:

$$V(G) = \text{Nodi Decisionali} + 1 = 8 + 1 = 9$$

Il numero ciclomatico indica che esistono **9 cammini linearmente indipendenti**. I cammini elementari del CFG sono 10, ma per la formula di McCabe solo 9 di essi sono linearmente indipendenti. I 10 cammini sono:

1. **N1(no) → N3(no) → fine**: nessun assioma, nessun nodo — termina.
2. **N1(sì) → N2(sì) → N1 → ...**: assioma attivo — aggiunto a `validNodes`.
3. **N1(sì) → N2(no) → N1 → ...**: assioma non attivo — ignorato.
4. **N1 → ... → N3(no) → fine**: ordine topologico vuoto.
5. **N1 → ... → N3(sì) → N4(sì) → N3**: nodo già valido — saltato.
6. **N1 → ... → N3(sì) → N4(no) → N5(no) → N3**: nodo assente — ignorato.
7. **N1 → ... → N3(sì) → N4(no) → N5(sì) → N6(sì) → N3**: è un assioma — saltato.
8. **N1 → ... → N3(sì) → N4(no) → N5(sì) → N6(no) → N7(sì) → N3**: tipo non visibile — saltato.
9. **N1 → ... → N3(sì) → N4(no) → N5(sì) → N6(no) → N7(no) → N8(sì) → N3**: nodo soddisfatto — aggiunto.
10. **N1 → ... → N3(sì) → N4(no) → N5(sì) → N6(no) → N7(no) → N8(no) → N3**: nodo non soddisfatto — ignorato.

Poiché il metodo itera su *molteplici* nodi all'interno di ciascun test case, un singolo test può esercitare più cammini contemporaneamente. Non è quindi necessario un test case per ogni cammino: è sufficiente che ogni cammino elementare venga percorso almeno una volta nell'insieme della suite. La tabella seguente mostra come i 5 test case coprono i cammini.

TC ID	Cammino CFG	Descrizione	Condizioni Logiche	Output atteso
TC_01	2, 3, 5, 7, 9	Catena completa: tutti i nodi soddisfatti	Assiomi attivi (2); altri assiomi non selezionati (3); nodi già validi saltati (5); assiomi nell'ordine topologico saltati (7); nodi soddisfatti aggiunti (9).	Tutti i nodi della catena in <code>validNodes</code> .
TC_02	1, 10	Nessun assioma attivo	Nessun assioma da iterare (1); nodi non soddisfatti perché mancano le premesse (10).	<code>validNodes.size = 0</code> .
TC_03	2, 3, 10	Dipendenza interrotta	Assioma attivo (2); altri assiomi non selezionati (3); nodo con dipendenza non soddisfatta (10).	Solo l'assioma attivo in <code>validNodes</code> .
TC_04	2, 3, 9	Prova OR: basta una dimostrazione	Assioma attivo (2); altri assiomi non selezionati (3); teorema soddisfatto via <code>some</code> (9).	Teorema in <code>validNodes</code> .
TC_05	2, 3, 6	Nodo assente nell'indice	Assiomi attivi (2); altri assiomi non selezionati (3); nodo non trovato nella mappa (6).	Nessun errore, valutazione prosegue.

Tabella 7: Copertura dei Cammini del CFG

Il cammino 4 non richiede un test dedicato perché l'ordine topologico non è mai vuoto nella pratica (il grafo contiene sempre almeno un nodo). Il cammino 8 (tipo non visibile) è esercitato implicitamente dalla presenza di nodi ausiliari (*sistema-axiomatico*, *matematico*, *lesson*) nell'ordine topologico. La copertura dei rami (*branch coverage*) è quindi completa per tutti i cammini raggiungibili.

6.2 Vitest - Test di Unità

Per il test di unità è stato utilizzato **Vitest**, il framework di testing nativo per progetti Vite/TypeScript. Si riporta di seguito uno dei test case della suite definita nel Capitolo 3, corrispondente a TC_04 (logica OR tra prove multiple):

```

1 import { describe, it, expect } from 'vitest';
2 import { Grafo } from '@entity/Grafo';
3
4 describe('Grafo.evaluate', () => {
5   it('handles theorem with multiple proofs (OR logic)', () => {
6     const multiProof = {
7       topologicalOrder: ['ax-1', 'ax-2', 'def-1', 'def-2', 'thm-1'],
8       nodes: {
9         'ax-1': { type: 'assioma', proofs: [], directDependencies: [] },
10        'ax-2': { type: 'assioma', proofs: [], directDependencies: [] },
11        'def-1': { type: 'definizione', proofs: [],
12          directDependencies: ['ax-1'] },
13        'def-2': { type: 'definizione', proofs: [],
14          directDependencies: ['ax-2'] },
15        'thm-1': { type: 'teorema',
16          proofs: [

```

```

17     { id: 'p1', dependencies: ['def-1'] },
18     { id: 'p2', dependencies: ['def-2'] },
19   ],
20   directDependencies: [],
21 },
22 },
23 };
24
25 const grafo = Grafo.from(multiProof);
26 const result = grafo.evaluate({ 'ax-1': true });
27 // Il teorema valido: la prima prova soddisfatta
28 expect(result.has('thm-1')).toBe(true);
29
30 const resultWithout = grafo.evaluate({});
31 // Nessun assioma attivo: il teorema non valido
32 expect(resultWithout.has('thm-1')).toBe(false);
33 });
34 });

```

Il test verifica che, in presenza di un teorema con due dimostrazioni alternative, sia sufficiente che una sola di esse abbia tutte le dipendenze soddisfatte perché il teorema risulti valido (logica OR). La suite completa comprende 11 test case che coprono i cammini indipendenti identificati dal CFG.

6.3 Test funzionale

La test suite definita nel Capitolo 3 è stata eseguita tramite Vitest. I test sono implementati nel file `tests/entity/grafito.test.ts` e possono essere eseguiti con il comando:

```
1 npx vitest run tests/entity/grafito.test.ts
```

La tabella seguente riporta per ciascun test case le classi di equivalenza coperte, l'output atteso, l'output effettivamente ottenuto e l'esito.

TC ID	Classi eq.	Descrizione	Output atteso	Output ottenuto	Esito
TC_01	A3, D1, I1	Catena completa valida	Tutti i nodi della catena risultano validi.	<code>validNodes</code> contiene tutti i 5 nodi della catena (ax-1, def-1, lem-1, thm-1, cor-1).	PASS
TC_02	A1, D3, I1	Nessun assioma attivo	Solo i nodi senza dipendenze sono validi.	<code>validNodes.size = 0</code> : nessun nodo risulta valido.	PASS
TC_03	A2, I1	Dipendenza interrotta	L'assioma attivo è valido; i nodi con dipendenza non soddisfatta non sono validi.	<code>validNodes</code> contiene solo ax-1; def-1, lem-1 e thm-1 non sono validi.	PASS
TC_04	A2, D2, I1	Logica OR tra prove	Il teorema è valido anche con una sola prova soddisfatta.	<code>validNodes.has('thm-1') = true</code> : la prima prova (p1) è soddisfatta.	PASS
TC_05	A3, I2	Nodo mancante nell'indice	Il nodo mancante viene ignorato; nessun errore.	Valutazione completa senza eccezioni; il nodo mancante è saltato.	PASS

Tabella 8: Risultati dei Test Funzionali

Tutti i test case hanno prodotto l'output atteso, confermando la correttezza dell'implementazione del metodo `Grafo.evaluate()` e la copertura dei cammini logici identificati dal CFG.